

Complete Explanation of the Paper

Why credit default prediction as the use case

Credit default prediction is one of the highest-stakes ML applications in finance. A bank needs to decide whether to approve a loan, at what interest rate, and for what amount — all in seconds. Machine learning models do this by learning patterns from historical loan outcomes. But regulators and borrowers both need to understand why a loan was denied. If the model says "denied" but can't explain which factors drove that decision, it violates fair lending laws and erodes trust. This makes credit risk the perfect domain for XAI research — the explanations aren't optional, they're legally required.

Why the LendingClub dataset

It's the largest publicly available real loan dataset with fully resolved outcomes — you know definitively whether each loan was repaid or defaulted. It spans 11 years (2007–2018) covering the Global Financial Crisis recovery, the zero-rate era, and the beginning of rate normalisation. This gives you three meaningfully different economic environments to test XSI across, without needing proprietary bank data.

Why XGBoost specifically

Three reasons that matter for the research, not just performance. First, TreeSHAP (SHAP's implementation for tree models) is exact for XGBoost — it computes the true Shapley values without approximation. For any other model type (neural networks, SVM), SHAP uses sampling-based approximations that introduce noise. If your stability metric (XSI) is built on SHAP values, you need those values to be exact, otherwise you can't distinguish real drift from approximation variance. Second, XGBoost is genuinely what banks use — making the research directly applicable to practice. Third, it handles class imbalance cleanly via `scale_pos_weight`, which is critical since only ~20% of loans default.

What SHAP actually computes and why it matters

SHAP is rooted in cooperative game theory from the 1950s. The original Shapley value answers this question: if several players cooperate to win a game and produce a collective outcome, how should the winnings be fairly distributed among them? The "players" in our case are features, the "game" is producing a prediction, and the "winnings" are the gap between the prediction for this specific loan and what the model would predict on average for any loan.

The Shapley value for feature i

i is computed by averaging the marginal contribution of feature i

i across all possible orderings in which features could be "introduced" to the prediction. If you have 34 features, there are $34!$ possible orderings — but TreeSHAP computes this exactly in polynomial time by exploiting the tree structure.

Why does this matter for our paper? Because it means SHAP values for a loan in 2010 and SHAP values for a similar loan in 2016 are computed on the same mathematical basis. The comparison is apples-to-apples. When SHAP says `fed_funds_rate` contributed +0.03 to the prediction in 2016 but only +0.001 in 2013, that difference is real and interpretable — not an artefact of the method.

What LIME does differently and why the comparison matters

LIME doesn't care about the global model. It only cares about this specific prediction. It creates hundreds of fake data points around the loan being explained, slightly perturbing each feature, runs the model on all of them, and then fits a simple linear model to the results. The coefficients of that linear model become the "explanation."

The problem is that LIME's result depends heavily on: how it creates the fake points (random sampling), how it discretizes continuous features (putting `int_rate` into bins), and how many fake points it creates. Two runs of LIME on the same loan with different random seeds will give different explanations. SHAP on the same loan twice will always give the same answer. The Spearman $\rho = 0.121$ between SHAP and LIME global rankings is a meaningful finding for the paper. It tells practitioners: if you're using LIME for regulatory explanations and SHAP for internal analysis, you're essentially using two different languages to describe the same model. Both might be "correct" in some technical sense but they're telling different stories. For temporal stability analysis specifically, LIME's stochasticity would contaminate the XSI signal — you'd never know if XSI dropped because explanations genuinely drifted or because LIME's random sampling happened to produce different results this window.

The rolling window design and why it's the right approach

A static model trained once and evaluated forever doesn't reflect reality. Banks retrain models regularly — quarterly or annually — as new loan data accumulates. Each retraining potentially produces a model with slightly different internal structure, and therefore different SHAP attributions. Our rolling windows simulate this: train on 12 months, evaluate on the next 3 months, advance by 3 months, repeat.

This design has a specific advantage for the paper's argument: because we retrain from scratch on each window, any changes in SHAP attributions between windows are due to real changes in the data distribution, not changes in the model's random seed or architecture. When we observe $XSI = 0.40$ in a window, it means the explanations genuinely became less consistent — not that we got unlucky with random initialisation.

The three components of XSI and why we need all three

Each component captures a different dimension of explanation stability. You need all three because each can fail independently.

Kendall τ (rank stability) asks: did the same features remain most important? It's possible for the ranking to be perfectly preserved (`sub_grade` is still #1, `int_rate` is still #2) but the magnitudes to shift dramatically — `sub_grade` used to contribute 0.10 to each prediction but now contributes 0.02. The ranking looks stable but the explanation is effectively different. Kendall τ alone would miss this.

Cosine similarity (directional stability) asks: do the attributions point in the same direction? It compares the full attribution vectors, not just their rankings. A cosine similarity of 0 means the new attribution vector is orthogonal to the old one — completely different pattern of which features matter. A cosine similarity of -1 means the model has literally reversed which features push toward and away from default. We floor at 0 because negative cosine means the explanations have fully reversed, which is the worst-case scenario and should contribute zero to XSI rather than a negative number.

PSI on SHAP distributions (distribution stability) asks: has the spread of SHAP values for each feature changed? PSI was originally invented to detect when a credit score distribution has shifted enough to warrant scorecard recalibration. We apply it to SHAP distributions instead of score distributions — a novel extension. A feature that used to have SHAP values

tightly clustered around 0.02 now has a bimodal distribution with values at -0.05 and +0.08 — the mean might be similar but the distribution has completely changed. This captures something neither Kendall τ nor cosine similarity would.

The weights ($\alpha=0.4$, $\beta=0.4$, $\gamma=0.2$) reflect the view that rank and directional stability are equally important and somewhat more central to the regulatory explanation use case, while distribution stability is informative but secondary. These are reasonable defaults and an ablation study would be a natural extension.

The HMM for regime detection and why it's necessary

Without regime labels, the XSI analysis is just a time series of numbers. With regime labels, you can ask the question that matters for practitioners: are explanations more unstable in bad economic times? The HMM answers this.

A Hidden Markov Model is appropriate here because economic regimes are not directly observable — you can't look up "which regime is today." You only observe noisy signals: Fed Funds Rate, VIX, default rates. The HMM assumes there's an underlying hidden state (Bull/Neutral/Bear-Crisis) that generates these observations, and the state transitions follow Markov dynamics (today's regime depends on yesterday's, not on what happened in 2010). The Baum-Welch algorithm learns both the emission distributions (what VIX/rate/default combinations are typical for each regime) and the transition probabilities from data. We post-process the state labels so that state 0 always means lowest default rate (Bull) and state 2 always means highest (Bear/Crisis). This ensures the regime labels are interpretable consistently across different runs.

EART and why 3.15 months matters

The central claim of EART is that explanation drift precedes accuracy degradation. When a model's explanations start shifting — the ranking of features changes, attribution magnitudes move — this is an early signal that the model's internal representation of risk has changed, even if its overall discrimination ability (AUC) has not yet deteriorated measurably.

Think of it this way: a model trained in 2013 learned that revolving utilisation is the #3 most important default predictor. By 2016 (post-tightening cycle), the macroeconomic context has changed. The model retrained on 2016 data now considers interest rate sensitivity more important than utilisation. The AUC might be virtually identical (the model is still predicting default well) but the explanation has shifted. A loan officer in 2016 who asks "why was this loan denied?" gets a fundamentally different answer than in 2013 — different features, different magnitudes, potentially different narrative. EART catches this the moment it happens.

The 3.15-month lead time is practically significant because quarterly is the cadence at which most banks conduct model validation reviews under SR 11-7. EART would have flagged the problem in the quarter it appeared, meaning the bank has time to investigate before the next review cycle. The AUC-based baseline would miss it for that cycle entirely.

Why XSI and AUC being uncorrelated ($r=0.204$) is a good result

This might seem counterintuitive. Wouldn't you want a metric that correlates with model quality? No — and here's why. If XSI perfectly correlated with AUC, it would add zero information. You could just watch AUC and get the same signal. The value of XSI is precisely that it measures something different from AUC. The $r=0.204$ tells you: knowing AUC tells you almost nothing about XSI and vice versa. A model can have high AUC and low XSI

(accurate predictions but unstable explanations — the dangerous EART zone). It can have low AUC and high XSI (poor predictions but consistent explanations — different kind of problem). EART is designed specifically to catch the first case, which accuracy monitoring would miss entirely.

The `fed_funds_rate` heatmap result as the narrative centrepiece

The most interpretable finding in the paper is the heatmap showing `fed_funds_rate` SHAP importance surging in 2016–2018. This is not a statistical artefact — it's an economically meaningful story. In 2013, interest rates had been near zero for years. The Fed Funds Rate was essentially a constant. As a predictive feature, a constant has no variance and therefore no predictive power — SHAP correctly assigns it near-zero importance because it barely varies across borrowers.

By 2016, the Fed started raising rates and different loan cohorts were originated at materially different rate environments. Suddenly, whether a loan was originated at the beginning vs. end of the hiking cycle mattered enormously for default risk. SHAP correctly picks this up — `fed_funds_rate` becomes one of the top features. This shift in feature importance is explanation drift. EART would have flagged it. A practitioner who saw the SHAP heatmap and the EART triggers together would have the information to understand why the model's explanation behaviour was changing — which is exactly the kind of interpretable governance output the paper promises.